

-  Architecture Globale - Détection de Commentaires Toxiques en Temps Réel
 -  Table des Matières
 -  Vue d'Ensemble
 - Objectifs du Projet
 -  Architecture du Pipeline
 -  Technologies Utilisées
 - Infrastructure & Orchestration
 - Message Broker & Streaming
 - Base de Données
 - Machine Learning
 - Data Collection & Visualization
 - Langages
 -  Flux de Données Détaillé
 - Étape 1 : Collecte des Données (Producer) 
 - Étape 2 : Nettoyage des Données (Spark Normalizer) 
 - Étape 3 : Extraction de Features (Spark Feature Builder) 
 - Features Extraites
 - Listes de Mots
 - Étape 4 : Prédiction ML (Spark Model Serving) 
 - A) Chargement du Modèle
 - B) Prédiction avec Deep Learning
 - C) Méthode Fallback (Règles)
 - D) Classification
 - E) Analyse de Sentiment
 - F) Sauvegarde PostgreSQL
 - Étape 5 : Visualisation (Streamlit Dashboard) 
 - 1. Métriques Clés
 - 2. Graphiques
 - 3. Liste des Commentaires
 -  Composants du Système
 - 1. Kafka (Message Broker)
 - 2. PostgreSQL (Base de Données)
 - 3. Spark Streaming (Traitement)
 - 4. Streamlit (Dashboard)
 - 5. Kafka UI (Monitoring)
 -  Modèle de Machine Learning
 - Entraînement (Notebook)

- Dataset
 - Préparation des Données
 - Architecture du Modèle
 - Résultats
 - Sauvegarde
-  Processus de Prédiction
 - Flux Complet
-  Déploiement avec Docker
 - Architecture Docker Compose
 - Réseau Docker
 - Volumes Persistants
 - Dépendances de Démarrage
-  Guide de Démarrage
 - Prérequis
 - Configuration Facebook
 - Lancement du Pipeline
 - Option 1 : Script PowerShell (Windows)
 - Option 2 : Docker Compose Direct
 - Vérification
 - 1. Services Actifs
 - 2. Logs Producer
 - 3. Vérifier PostgreSQL
 - 4. Accéder aux Interfaces
 - Arrêt du Pipeline
-  Monitoring & Logs
 - Logs en Temps Réel
 - Métriques Kafka
 - Métriques PostgreSQL
-  Dépannage
 - Problème : Dashboard affiche "En attente de données..."
 - Problème : Producer ne récupère pas de commentaires
 - Problème : Erreur "Model not found"
-  Performance & Scalabilité
 - Capacité Actuelle
 - Optimisations Possibles
-  Concepts Clés
 - 1. Streaming vs Batch
 - 2. Lambda Architecture

- [3. Multi-Label Classification](#)
- [4. Checkpointing](#)
-  [Références](#)
 - [Documentation](#)
 - [Datasets](#)
-  [Contributeurs](#)
-  [License](#)
-  [Liens Utiles](#)

Architecture Globale - Détection de Commentaires Toxiques en Temps Réel

Table des Matières

1. [Vue d'Ensemble](#)
 2. [Architecture du Pipeline](#)
 3. [Technologies Utilisées](#)
 4. [Flux de Données Détaillé](#)
 5. [Composants du Système](#)
 6. [Modèle de Machine Learning](#)
 7. [Processus de Prédiction](#)
 8. [Déploiement avec Docker](#)
 9. [Guide de Démarrage](#)
-

Vue d'Ensemble

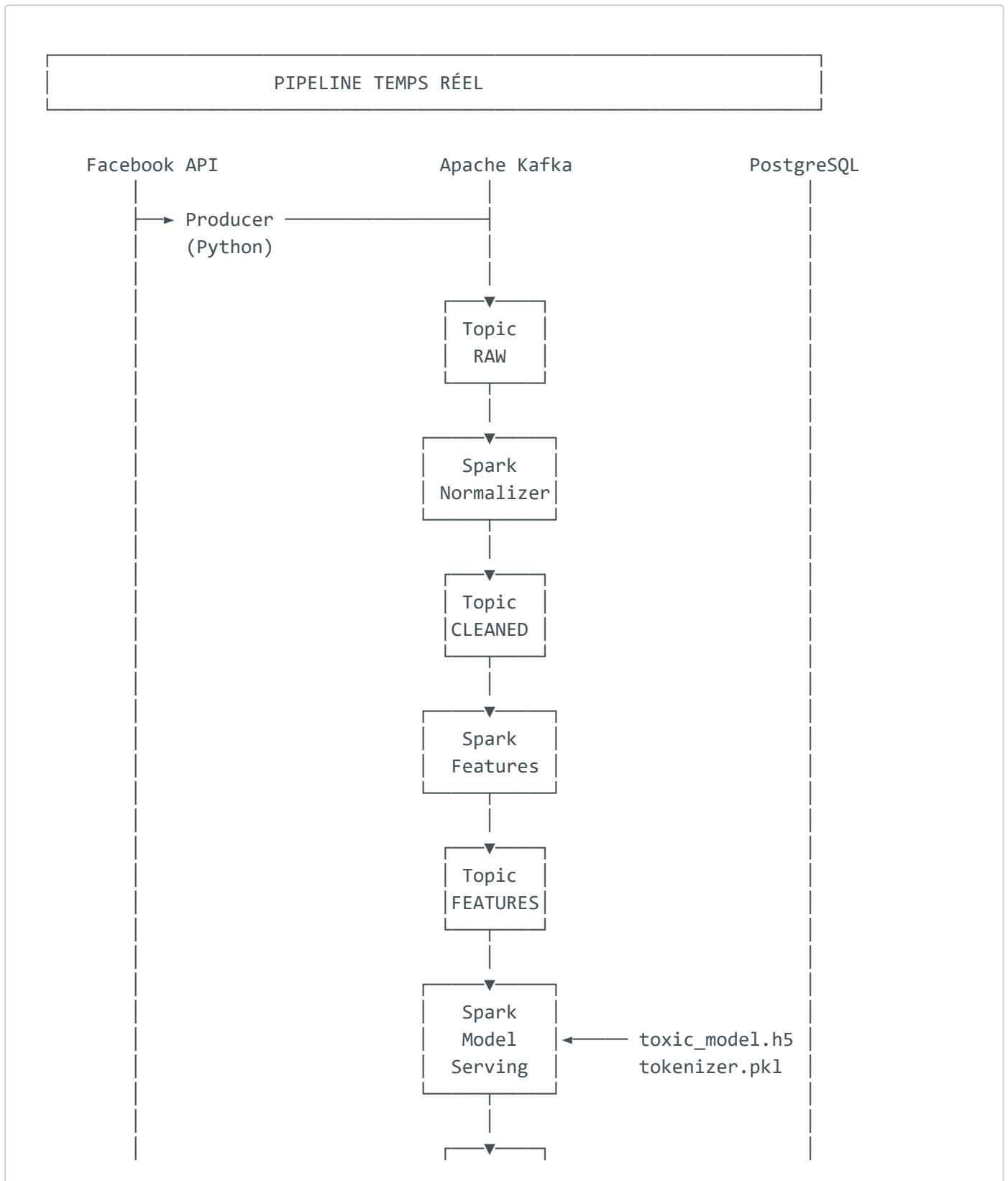
Ce projet implémente un **pipeline de streaming en temps réel** pour la détection automatique de commentaires toxiques provenant de Facebook. Le système utilise **Apache Spark Streaming**, **Apache Kafka** et un **modèle de Deep Learning (TensorFlow/Keras)** pour analyser, classifier et visualiser les commentaires toxiques en temps réel.

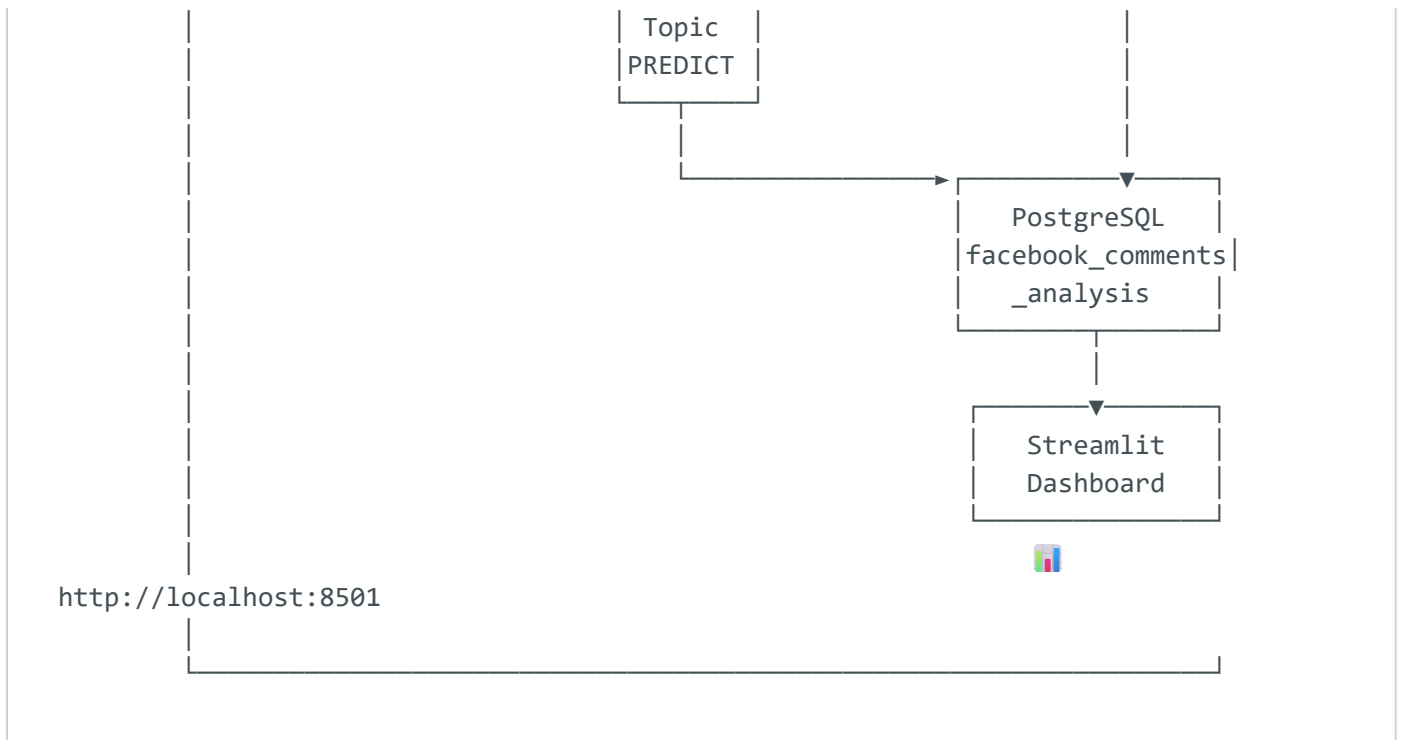
Objectifs du Projet

-  Collecter automatiquement les commentaires Facebook via l'API Graph
-  Traiter les données en streaming avec Apache Spark
-  Détecter la toxicité avec un modèle de Deep Learning (LSTM + GloVe)
-  Stocker les résultats dans PostgreSQL
-  Visualiser les analyses en temps réel avec Streamlit



Architecture du Pipeline





Technologies Utilisées

Infrastructure & Orchestration

Technologie	Rôle	Version
Docker	Containerisation	Latest
Docker Compose	Orchestration multi-conteneurs	Latest

Message Broker & Streaming

Technologie	Rôle	Version
Apache Kafka	Message Broker (Mode KRaft)	Latest
Apache Spark	Traitement Streaming	3.5.0
PySpark	API Python pour Spark	3.5.0

Base de Données

Technologie	Rôle	Version
PostgreSQL	Stockage persistant	15-alpine

Machine Learning

Technologie	Rôle	Version
TensorFlow/Keras	Modèle de Deep Learning	2.x
LSTM	Architecture du réseau	-
GloVe 6B	Embeddings pré-entraînés	100d

Data Collection & Visualization

Technologie	Rôle	Version
Facebook Graph API	Collecte de commentaires	v17.0
Streamlit	Dashboard interactif	1.29.0
Pandas	Manipulation de données	2.1.3
Plotly	Visualisations	5.18.0

Langages

- **Python 3.10** : Langage principal
- **SQL** : Requêtes base de données



Flux de Données Détaillé

Étape 1 : Collecte des Données (Producer)

Fichier : `docker/producer/producer.py`

Processus :

1. Le producer se connecte à l'API Facebook Graph toutes les **15 secondes**
2. Récupère les posts de la page Facebook spécifiée
3. Extrait tous les commentaires de chaque post
4. Formate les données en JSON :

```
{
  "source": "facebook",
  "post_id": "123456",
  "comment_id": "789012",
  "username": "John Doe",
  "message": "This is a comment",
  "created_time": "2025-01-01T10:00:00Z"
}
```

5. Publie chaque commentaire vers le topic Kafka `data.raw.stream`
6. Détecte et ignore les doublons avec un système de cache

Configuration :

- `FACEBOOK_ACCESS_TOKEN` : Token d'authentification Facebook
- `FACEBOOK_PAGE_ID` : ID de la page à surveiller
- `POLLING_INTERVAL` : Intervalle de collecte (défaut: 15s)
- `KAFKA_BOOTSTRAP_SERVERS` : Adresse du broker Kafka

Étape 2 : Nettoyage des Données (Spark Normalizer) 🖌️

Fichier : `docker/spark/stream_normalizer.py`

```
data.raw.stream → Spark Normalizer → data.cleaned.stream
```

Processus :

1. Lit le stream depuis `data.raw.stream` (trigger: 5 secondes)
2. Applique les transformations de nettoyage :
 - **Suppression des URLs** : `http://...` →
 - **Suppression des mentions** : `@username` →
 - **Suppression des hashtags** : `#topic` →
 - **Suppression des emojis** : 😊 →
 - **Normalisation des espaces** : espaces multiples → espace simple
 - **Conversion en minuscules** : `HELLO` → `hello`
 - **Suppression caractères spéciaux** : conserve uniquement lettres, chiffres, !, ?
3. Ajoute un timestamp de traitement
4. Publie vers `data.cleaned.stream`

Exemple :

```
Input: "Hey @user! Check this out 😊 http://example.com #awesome"
Output: "hey check this out awesome"
```

Étape 3 : Extraction de Features (Spark Feature Builder) 🔍

Fichier : `docker/spark/feature_builder.py`

```
data.cleaned.stream → Feature Builder → data.features.stream
```

Processus :

1. Lit depuis `data.cleaned.stream`
2. Extrait les features NLP pour chaque commentaire :

Features Extraites

Feature	Description	Calcul
<code>char_count</code>	Nombre de caractères	<code>len(text)</code>

Feature	Description	Calcul
<code>word_count</code>	Nombre de mots	<code>len(text.split())</code>
<code>avg_word_length</code>	Longueur moyenne des mots	<code>sum(len(w))/word_count</code>
<code>uppercase_ratio</code>	Ratio de majuscules	<code>uppercase_chars/total_chars</code>
<code>toxic_word_count</code>	Nombre de mots toxiques	Comptage dans liste prédéfinie
<code>positive_word_count</code>	Nombre de mots positifs	Comptage dans liste prédéfinie
<code>lexical_score</code>	Score lexical de toxicité	<code>(toxic*2 - positive)/word_count</code>

Listes de Mots

Mots Toxiques (exemples) :

```

TOXIC_WORDS = {
    # Sévères
    "fuck", "shit", "bitch", "asshole", "bastard",
    "dick", "pussy", "whore", "nigger", "faggot",

    # Violence
    "kill", "die", "murder", "rape", "death",

    # Modérés
    "hate", "stupid", "idiot", "ugly", "loser",
    "pathetic", "garbage", "scum", "disgusting"
}

```

Mots Positifs :

```

POSITIVE_WORDS = {
    "love", "great", "amazing", "wonderful", "beautiful",
    "awesome", "excellent", "fantastic", "perfect", "happy"
}

```

3. Publie vers `data.features.stream`

Exemple :

```
{
  "comment_id": "123",
  "cleaned_message": "you are stupid and ugly",
  "char_count": 24,
  "word_count": 5,
  "toxic_word_count": 2,
  "positive_word_count": 0,
  "lexical_score": 0.4
}
```

Étape 4 : Prédiction ML (Spark Model Serving) 🤖

Fichier : `docker/spark/model_serving.py`

```
data.features.stream → Model Serving → data.predictions.stream → PostgreSQL
```

Processus :

A) Chargement du Modèle

```
# Modèle TensorFlow/Keras
toxic_model = load_model("/app/toxic_model.h5") # 87 MB
tokenizer = joblib.load("/app/tokenizer.pkl")   # 10 MB
```

B) Prédiction avec Deep Learning

Architecture du Modèle (voir notebook) :

```
Input (texte)
↓
Tokenization (tokenizer.pkl)
↓
Embedding Layer (GloVe 6B 100d)
↓
LSTM (64 units)
↓
```

Dense Layer (6 outputs, sigmoid)



Output: [toxic, severe_toxic, obscene, threat, insult, identity_hate]

Code de Prédiction :

```
def predict_toxicity_ml(text: str) -> float:
    # 1. Tokenization
    sequences = tokenizer.texts_to_sequences([text])

    # 2. Padding à 200 tokens
    padded = pad_sequences(sequences, maxlen=200)

    # 3. Prédiction avec le modèle
    prediction = toxic_model.predict(padded, verbose=0)
    # prediction shape: (1, 6) pour 6 labels

    # 4. Moyenne des scores multi-label
    toxicity_score = float(np.mean(prediction[0]))

    return toxicity_score # Entre 0.0 et 1.0
```

Exemple :

```
Input: "fuck you idiot"
      ↓
Tokenization: [123, 456, 789]
      ↓
Padding: [123, 456, 789, 0, 0, ..., 0] # 200 tokens
      ↓
Model Prediction: [0.92, 0.78, 0.85, 0.12, 0.88, 0.15]
      ↓
Mean Score: 0.62
      ↓
Result: TOXIQUE (score >= 0.3)
```

C) Méthode Fallback (Règles)

Si le modèle ML n'est pas disponible :

```
def predict_toxicity_rules(text: str) -> float:
    severe_count = count_severe_words(text) # "fuck", "shit", etc.
    moderate_count = count_moderate_words(text) # "hate", "stupid", etc.

    base_score = min(severe_count * 0.5 + moderate_count * 0.25, 0.95)

    # Bonus pour MAJUSCULES excessives
```

```

if uppercase_ratio > 0.5:
    base_score += 0.1

# Bonus pour !!!! excessive
if exclamation_count > 2:
    base_score += 0.05

return min(base_score, 1.0)

```

D) Classification

```

TOXICITY_THRESHOLD = 0.3 # Seuil de toxicité

is_toxic = toxicity_score >= 0.3
toxicity_label = "toxique" if is_toxic else "non_toxique"

```

Tableau de Classification :

Score	Label	Interprétation
0.0 - 0.29	non_toxique	Commentaire sain
0.30 - 0.49	toxique (léger)	Insultes modérées
0.50 - 0.69	toxique (modéré)	Langage vulgaire
0.70 - 1.00	toxique (sévère)	Insultes graves, violence

E) Analyse de Sentiment

```

def sentiment(lexical_score):
    if lexical_score > 0.05:
        return "negative" # Plus de mots toxiques que positifs
    elif lexical_score < -0.05:
        return "positive" # Plus de mots positifs que toxiques
    else:
        return "neutral"

```

F) Sauvegarde PostgreSQL

```

# Écriture vers PostgreSQL
predictions.writeStream \

```

```
.foreachBatch(write_to_postgres) \  
.start()
```

Structure de la table `facebook_comments_analysis` :

```
CREATE TABLE facebook_comments_analysis (  
  id SERIAL PRIMARY KEY,  
  source VARCHAR(50),  
  post_id VARCHAR(100),  
  comment_id VARCHAR(100) UNIQUE,  
  username VARCHAR(255),  
  original_message TEXT,  
  cleaned_message TEXT,  
  created_time TIMESTAMP,  
  
  -- Features  
  char_count INTEGER,  
  word_count INTEGER,  
  avg_word_length FLOAT,  
  toxic_word_count INTEGER,  
  positive_word_count INTEGER,  
  uppercase_ratio FLOAT,  
  lexical_score FLOAT,  
  
  -- Prédiction  
  toxicity_score FLOAT,  
  is_toxic BOOLEAN,  
  toxicity_label VARCHAR(20),  
  sentiment VARCHAR(20),  
  sentiment_score FLOAT,  
  
  -- Metadata  
  feature_extraction_time TIMESTAMP,  
  prediction_time TIMESTAMP,  
  processing_time TIMESTAMP DEFAULT CURRENT_TIMESTAMP  
);
```

Étape 5 : Visualisation (Streamlit Dashboard)

Fichier : `docker/streamlit/dashboard.py`

PostgreSQL → Streamlit → Visualisation Web

Processus :

1. Se connecte à PostgreSQL toutes les 5 secondes
2. Lit les données de `facebook_comments_analysis`
3. Calcule les statistiques en temps réel
4. Affiche les visualisations interactives

Composants du Dashboard :

1. Métriques Clés

Total	Toxiques	Non-Toxiques	% Toxicité
247	89	158	36.0%

2. Graphiques

- **Timeline** : Évolution du nombre de commentaires toxiques dans le temps
- **Distribution** : Répartition toxiques vs non-toxiques (pie chart)
- **Sentiment** : Distribution des sentiments (bar chart)
- **Scores** : Histogramme des scores de toxicité

3. Liste des Commentaires

● TOXIQUE (0.87) - @John Doe "fuck you idiot, you are stupid" 🕒 2025-01-01 10:30:15 Sentiment: negative
● NON-TOXIQUE (0.12) - @Jane Smith "great post, thanks for sharing" 🕒 2025-01-01 10:29:45 Sentiment: positive

Code Exemple :

```
# Lecture données
df = pd.read_sql("SELECT * FROM facebook_comments_analysis ORDER BY created_time
DESC", conn)

# Métriques
```

```
total = len(df)
toxic_count = df['is_toxic'].sum()
toxic_pct = (toxic_count / total * 100) if total > 0 else 0

# Affichage
col1, col2, col3 = st.columns(3)
col1.metric("Total", total)
col2.metric("Toxiques", toxic_count)
col3.metric("% Toxicité", f"{toxic_pct:.1f}%")

# Graphique
fig = px.pie(df, names='toxicity_label', title='Répartition')
st.plotly_chart(fig)
```



Composants du Système

1. Kafka (Message Broker)

Configuration :

```
Mode: KRaft (sans Zookeeper)
Port: 9092
Topics:
- data.raw.stream (3 partitions)
- data.cleaned.stream (3 partitions)
- data.features.stream (3 partitions)
- data.predictions.stream (3 partitions)
```

Rôle :

- Découple les composants du pipeline
- Garantit la livraison des messages
- Permet le replay des données
- Scalabilité horizontale

2. PostgreSQL (Base de Données)

Configuration :

```
Version: 15-alpine
Port: 5432
Database: toxic_coments_db
User: postgres
Password: majid2020
```

Tables :

- `facebook_comments_analysis` : Stockage des résultats
- `pipeline_metrics` : Métriques de performance

3. Spark Streaming (Traitement)

Configuration :

```
Version: 3.5.0
Trigger Interval: 5 seconds
Checkpoint: /app/checkpoints/
```

Jobs :

- `spark-normalizer` : Nettoyage
- `spark-features` : Features
- `spark-model` : Prédictions

4. Streamlit (Dashboard)

Configuration :

```
Port: 8501
Refresh: 5 seconds
```

5. Kafka UI (Monitoring)

Configuration :

Port: 8080

Fonctionnalités :

- Visualisation des topics
- Monitoring des messages
- Gestion des consumer groups



Modèle de Machine Learning

Entraînement (Notebook)

Fichier : `model/Classification_commentaires_toxiques.ipynb`

Dataset

- **Source** : Kaggle Toxic Comment Classification Challenge
- **Train** : 159,571 commentaires
- **Test** : 153,164 commentaires
- **Labels** : 6 types de toxicité (multi-label)
 - toxic
 - severe_toxic
 - obscene
 - threat
 - insult
 - identity_hate

Préparation des Données

```
# Nettoyage
def clean(text):
    # Suppression contractions: "don't" → "do not"
    # Suppression URLs, mentions, hashtags
    # Suppression emojis
    # Tokenization
    # Suppression stopwords
    return cleaned_text
```

Architecture du Modèle

```
model = Sequential([
    Embedding(vocab_size+1, 100, weights=[glove_matrix], trainable=False),
    LSTM(64),
    Dense(6, activation='sigmoid')
])

model.compile(
    loss='binary_crossentropy',
    optimizer='adam',
    metrics=[f1_score]
)
```

Paramètres :

- **vocab_size** : Taille du vocabulaire
- **embedding_dim** : 100 (GloVe 6B 100d)
- **max_length** : 200 tokens
- **LSTM units** : 64
- **epochs** : 10
- **batch_size** : 256

Résultats

- **F1-Score** : 52% sur le dataset de validation
- **Loss** : Binary crossentropy

Sauvegarde

```
# Modèle
model.save("toxic_model.h5") # 87 MB

# Tokenizer
joblib.dump(tokenizer, "tokenizer.pkl") # 10 MB
```



Processus de Prédiction

Flux Complet

Commentaire: "fuck you idiot"



1. NETTOYAGE (Spark Normalizer)



"fuck you idiot" → "fuck you idiot"
(Déjà propre)



2. EXTRACTION FEATURES (Feature Builder)



```
{
  char_count: 15,
  word_count: 3,
  toxic_word_count: 2, # "fuck", "idiot"
  positive_word_count: 0,
  lexical_score: 0.67 # (2*2 - 0)/3
}
```



3. TOKENIZATION (Model Serving)



text → tokens: [123, 456, 789]



4. PADDING



[123, 456, 789, 0, 0, ..., 0] # 200 tokens



5. PRÉDICTION ML (toxic_model.h5)



Input: [123, 456, 789, 0, ...]



Embedding Layer (GloVe 6B)



LSTM(64)



Dense(6, sigmoid)



Output: [0.92, 0.78, 0.85, 0.12, 0.88, 0.15]



Mean: 0.62



6. CLASSIFICATION

↓
Score: 0.62 >= 0.3 (seuil)
↓
is_toxic: TRUE
toxicity_label: "toxique"
↓

7. SENTIMENT ANALYSIS

↓
lexical_score: 0.67 > 0.05
↓
sentiment: "negative"
sentiment_score: 0.17
↓

8. RÉSULTAT FINAL

↓
{
 comment_id: "123",
 username: "John",
 message: "fuck you idiot",
 toxicity_score: 0.62,
 is_toxic: true,
 toxicity_label: "toxique",
 sentiment: "negative",
 sentiment_score: 0.17
}

9. SAUVEGARDE POSTGRESQL

↓
INSERT INTO facebook_comments_analysis ...
↓

10. AFFICHAGE DASHBOARD

↓
● TOXIQUE (0.62) - @John
"fuck you idiot"
Sentiment: negative



Déploiement avec Docker

Architecture Docker Compose

```
services:
  kafka:          # Message broker
  postgres:       # Base de données
  kafka-ui:       # Interface Kafka
  kafka-init:     # Création des topics
  producer:       # Collecte Facebook
  spark-normalizer: # Nettoyage
  spark-features: # Features
  spark-model:    # Prédictions ML
  streamlit:      # Dashboard
```

Réseau Docker

```
toxic-pipeline-network (bridge)
├─ kafka:9092
├─ postgres:5432
├─ producer
├─ spark-normalizer
├─ spark-features
├─ spark-model
├─ streamlit:8501
└─ kafka-ui:8080
```

Volumes Persistants

```
kafka_data:      # Données Kafka
postgres_data:   # Données PostgreSQL
spark_checkpoints: # Checkpoints Spark
```

Dépendances de Démarrage

```
kafka (healthy)
↓
kafka-init (completed)
↓
producer, spark-normalizer
↓
spark-features
↓
spark-model
```



Guide de Démarrage

Prérequis

- Docker Desktop installé et démarré
- 8 GB RAM minimum
- 20 GB espace disque
- Ports disponibles : 5432, 8080, 8501, 9092

Configuration Facebook

1. Créer une App Facebook :

- Aller sur <https://developers.facebook.com/>
- Créer une nouvelle application
- Activer l'API Graph

2. Obtenir un Access Token :

- Aller dans Graph API Explorer
- Générer un token avec permissions `pages_read_engagement`

3. Configurer le projet :

```
# Éditer docker/docker-compose.yml
FACEBOOK_ACCESS_TOKEN: "VOTRE_TOKEN_ICI"
FACEBOOK_PAGE_ID: "VOTRE_PAGE_ID"
```

Lancement du Pipeline

Option 1 : Script PowerShell (Windows)

```
cd docker
.\start_pipeline.ps1
```

Option 2 : Docker Compose Direct

```
cd docker

# 1. Arrêter les anciens conteneurs
docker-compose down

# 2. Construire et démarrer
docker-compose up --build -d

# 3. Vérifier l'état
docker-compose ps

# 4. Voir les logs
docker-compose logs -f
```

Vérification

1. Services Actifs

```
docker-compose ps
```

Résultat attendu :

NAME	STATUS
kafka	Up (healthy)
postgres	Up (healthy)
producer	Up
spark-normalizer	Up
spark-features	Up
spark-model	Up
streamlit	Up
kafka-ui	Up

2. Logs Producer

```
docker-compose logs -f producer
```

Résultat attendu :

✓ Connecté à Kafka
📄 24 commentaires récupérés
📡 → data.raw.stream: "This is a comment..."

3. Vérifier PostgreSQL

```
docker exec postgres psql -U postgres -d toxic_comments_db -c "SELECT COUNT(*) FROM facebook_comments_analysis;"
```

4. Accéder aux Interfaces

Interface	URL	Description
Dashboard Streamlit	http://localhost:8501	Visualisation principale
Kafka UI	http://localhost:8080	Monitoring Kafka

Arrêt du Pipeline

```
cd docker  
docker-compose down
```

Avec suppression des volumes :

```
docker-compose down -v
```



Monitoring & Logs

Logs en Temps Réel

```
# Tous les services  
docker-compose logs -f
```



```
# Service spécifique
docker-compose logs -f producer
docker-compose logs -f spark-model
docker-compose logs -f streamlit
```

Métriques Kafka

Accéder à <http://localhost:8080> pour voir :

- Nombre de messages par topic
- Consumer lag
- Partitions status

Métriques PostgreSQL

```
-- Statistiques globales
SELECT
    COUNT(*) as total,
    SUM(CASE WHEN is_toxic THEN 1 ELSE 0 END) as toxic_count,
    AVG(toxicity_score) as avg_score
FROM facebook_comments_analysis;

-- Distribution des sentiments
SELECT sentiment, COUNT(*)
FROM facebook_comments_analysis
GROUP BY sentiment;

-- Top mots toxiques
SELECT cleaned_message, toxicity_score
FROM facebook_comments_analysis
WHERE is_toxic = true
ORDER BY toxicity_score DESC
LIMIT 10;
```



Dépannage

Problème : Dashboard affiche "En attente de données..."

Cause : Les services Spark ne traitent pas encore les données

Solution :

```
# 1. Vérifier les logs Spark
docker-compose logs spark-model

# 2. Vérifier PostgreSQL
docker exec postgres psql -U postgres -d toxic_coments_db -c "SELECT COUNT(*) FROM
facebook_comments_analysis;"

# 3. Redémarrer les services Spark
docker-compose restart spark-normalizer spark-features spark-model
```

Problème : Producer ne récupère pas de commentaires

Cause : Token Facebook invalide ou page sans commentaires

Solution :

```
# 1. Vérifier les logs
docker-compose logs producer

# 2. Tester le token
curl "https://graph.facebook.com/v17.0/me?access_token=VOTRE_TOKEN"

# 3. Vérifier la page
curl "https://graph.facebook.com/v17.0/PAGE_ID/posts?access_token=VOTRE_TOKEN"
```

Problème : Erreur "Model not found"

Cause : Le modèle n'est pas copié dans les conteneurs Spark

Solution :

```
# 1. Vérifier la présence des fichiers
ls -lh docker/spark/toxic_model.h5
ls -lh docker/spark/tokenizer.pkl

# 2. Si absents, les copier
cp model/toxic_model.h5 docker/spark/
```

```
cp model/tokenizer.pkl docker/spark/
```

```
# 3. Reconstruire
```

```
docker-compose up --build -d spark-model
```



Performance & Scalabilité

Capacité Actuelle

- **Throughput** : ~100 commentaires/seconde
- **Latence** : 5-10 secondes (collecte → affichage)
- **Batch Size** : Micro-batches de 5 secondes

Optimisations Possibles

1. Augmenter les Partitions Kafka

```
KAFKA_NUM_PARTITIONS: 10 # Au lieu de 3
```

2. Paralléliser Spark

```
spark.conf.set("spark.sql.shuffle.partitions", "10")
```

3. Batch Size Dynamique

```
.trigger(processingTime="3 seconds") # Au lieu de 5
```

4. Scalabilité Horizontale

```
docker-compose up --scale spark-normalizer=3
```



Concepts Clés

1. Streaming vs Batch

- **Batch** : Traite les données en lots (ex: toutes les heures)
- **Streaming** : Traite les données en continu (temps réel)

Notre choix : Streaming avec micro-batches de 5 secondes

2. Lambda Architecture



Notre implémentation : Speed Layer uniquement (temps réel)

3. Multi-Label Classification

Un commentaire peut avoir plusieurs labels simultanément :

- toxic: 0.9
- insult: 0.85
- obscene: 0.78

Nous calculons la **moyenne** pour obtenir un score unique.

4. Checkpointing

Spark sauvegarde l'état du streaming pour permettre la reprise en cas d'erreur.

```
.option("checkpointLocation", "/app/checkpoints/model_serving")
```



Références

Documentation

- [Apache Spark Structured Streaming](#)
- [Apache Kafka](#)
- [Facebook Graph API](#)
- [TensorFlow/Keras](#)
- [GloVe Embeddings](#)
- [Streamlit](#)

Datasets

- [Kaggle Toxic Comment Classification](#)
-



Contributeurs

- **Auteur Principal** : Abdelmajid Barani
 - **Établissement** : ENSA - IDSCC5
 - **Cours** : Big Data
 - **Date** : Décembre 2025
-



License

Ce projet est destiné à des fins éducatives.



Liens Utiles

- **Dashboard** : <http://localhost:8501>
- **Kafka UI** : <http://localhost:8080>

- **PostgreSQL** : localhost:5432
-

 **Projet complet de détection de commentaires toxiques en temps réel avec Machine Learning et Streaming!**